Paper Review • arXiv 2610.37393

Learning Quality-Controllable Robot Policies with Iterative Hindsight Relabeling

Tunable robot execution quality, refined by iterative hindsight relabeling

Sinjae Kang, Dong Kyu Shin, Jaehong Kim, Kamin Lee, Hyoeun Kim, Kaixin Wang, Li Zhao, Seonghyeon Ye, Kimin Lee

KAIST • LGE • Microsoft Research

First submitted: 2026-10-27 Project page: sinnnj.github.io/qcp

Quality-Conditioned BC

Hindsight Relabel Loop

No human quality labels

π0.5 real-robot validation

SUMMARY

One-Sentence Summary

Encode execution quality as **quantile labels** that condition the policy, selectable by level at inference; then the loop "**collect at top level → keep successes → relabel by measured quality → retrain from scratch**" pushes controllable quality beyond the initial demos **without human labels**, raising quality and success rate together.

8 / 8

Sim task × attribute combos
Best on all quality metrics · Table 1

≈1.6

Smoothness / joint path length
Shift W1/IQR · Fig. 4

+59%

Real conveyor throughput
8.2 → 13.0 cans/min · Fig. 7b

42.0%

Real-robot wine-tray SR
BC only 16.0% · Fig. 7a

MOTIVATION

BC learns to succeed, not how—yet deployment needs tunable quality

1

BC treats all demos alike

All successful demos are treated as equally good → reproduces an “average” mix of fast/slow, smooth/jerky behavior that cannot be steered

2

Needs vary by scenario

Conveyors prize speed; carrying a tray of wine glasses prizes stable transport

3

Existing methods hit a ceiling

SORL / SCIQL etc. train only on static demos: the controllable range is capped by the demos, and quality trades off against success

Goals of This Work

01 Controllable at inference

Specify levels for one quality attribute or a combination on demand

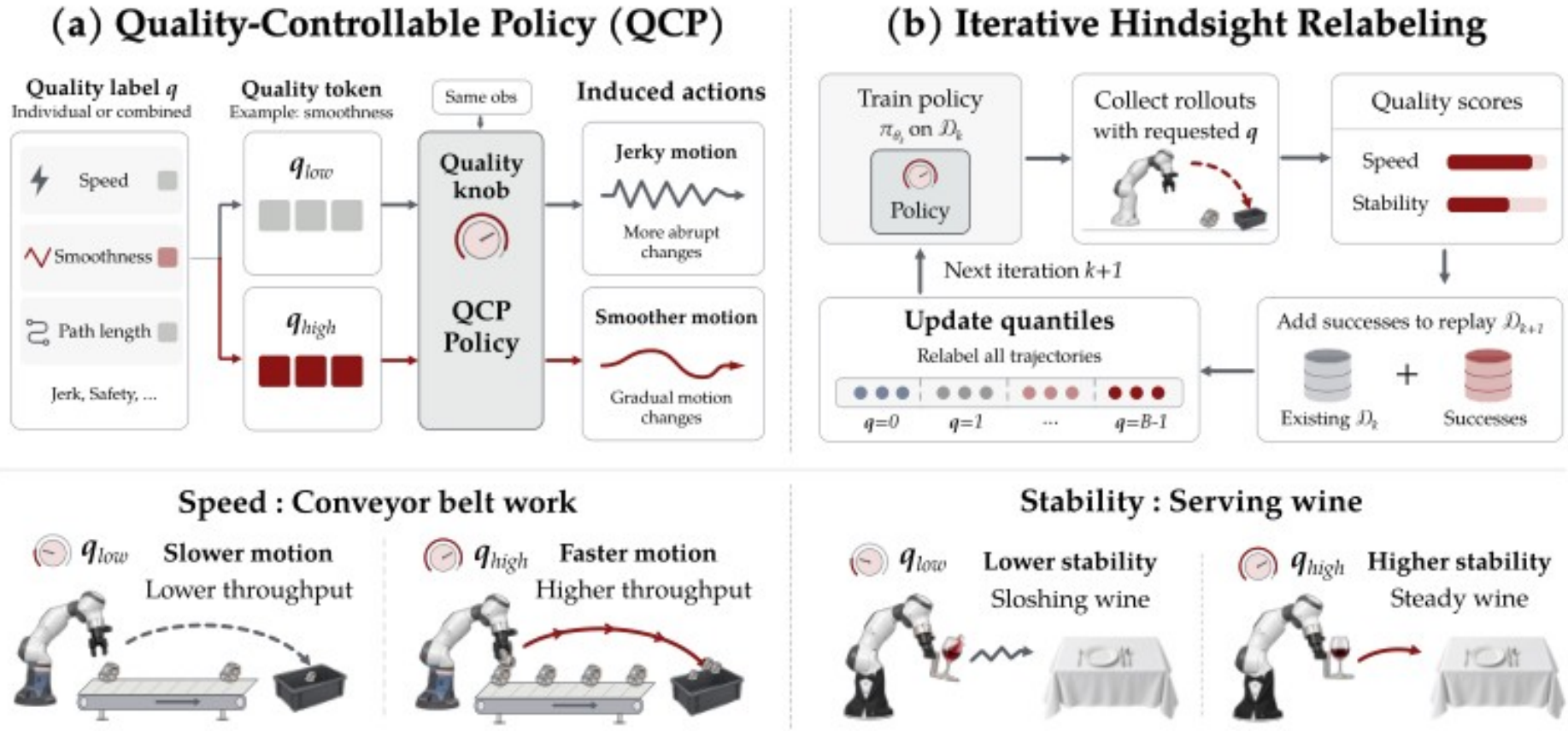
02 Ever-expanding control range

Surpass initial demos via online rollouts and policy updates

03 Zero human quality labels

Quality computed automatically by rule-based scoring functions

QCP overview: quality label as a knob; iterative relabeling raises the scale



(a) Quality-Controllable Policy label q → quality token → different-quality actions from one observation (jerky vs. smooth)

(b) Iterative Hindsight Relabeling train → collect at requested level → score → store successes → update quantiles, relabel all

QCP: quantile labels make quality a conditioning input for BC

① Rule-based score m(τ)

Speed 1/T, action jerk, joint path length, end-effector tilt… all oriented so that higher is better

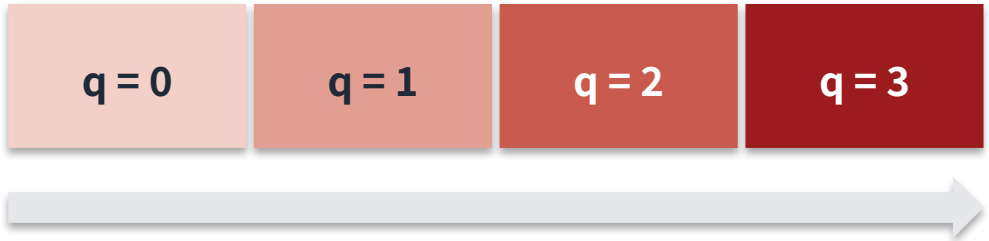
② Equal-frequency quantization into labels

Split by empirical CDF into B=4 bins (quartiles): q ∈ {0,1,2,3}; all action chunks of a trajectory share one label

③ Conditional flow matching

$\pi_\theta(A_t \mid h_t, q)$. MLP: q → embedding concatenated with observation encoding; VLA (π0.5): prompt includes "quality level: 3"

Equal-frequency bins (≈1/4 of trajectories each)



Core idea: condition, don't filter

Low-quality successful trajectories are not discarded—they still teach the policy how to complete the task, just under a low label.

→ Quality and success rate can **both** improve

Iterative relabeling: own successful rollouts keep raising the top-level bar

① Collect
Run π_k at top level q*=3 (300 per round in sim)

② Filter
Keep only successes (adding failures hurts)

③ Relabel
Store by measured score; recompute all quantiles on D_{k+1}

④ Retrain
Train π_{k+1} from scratch (MLP: random init / VLA: from pretrained)

↺ Round k+1: high-quality rollouts accumulate → top threshold rises; original demos keep the low end → controllable range widens

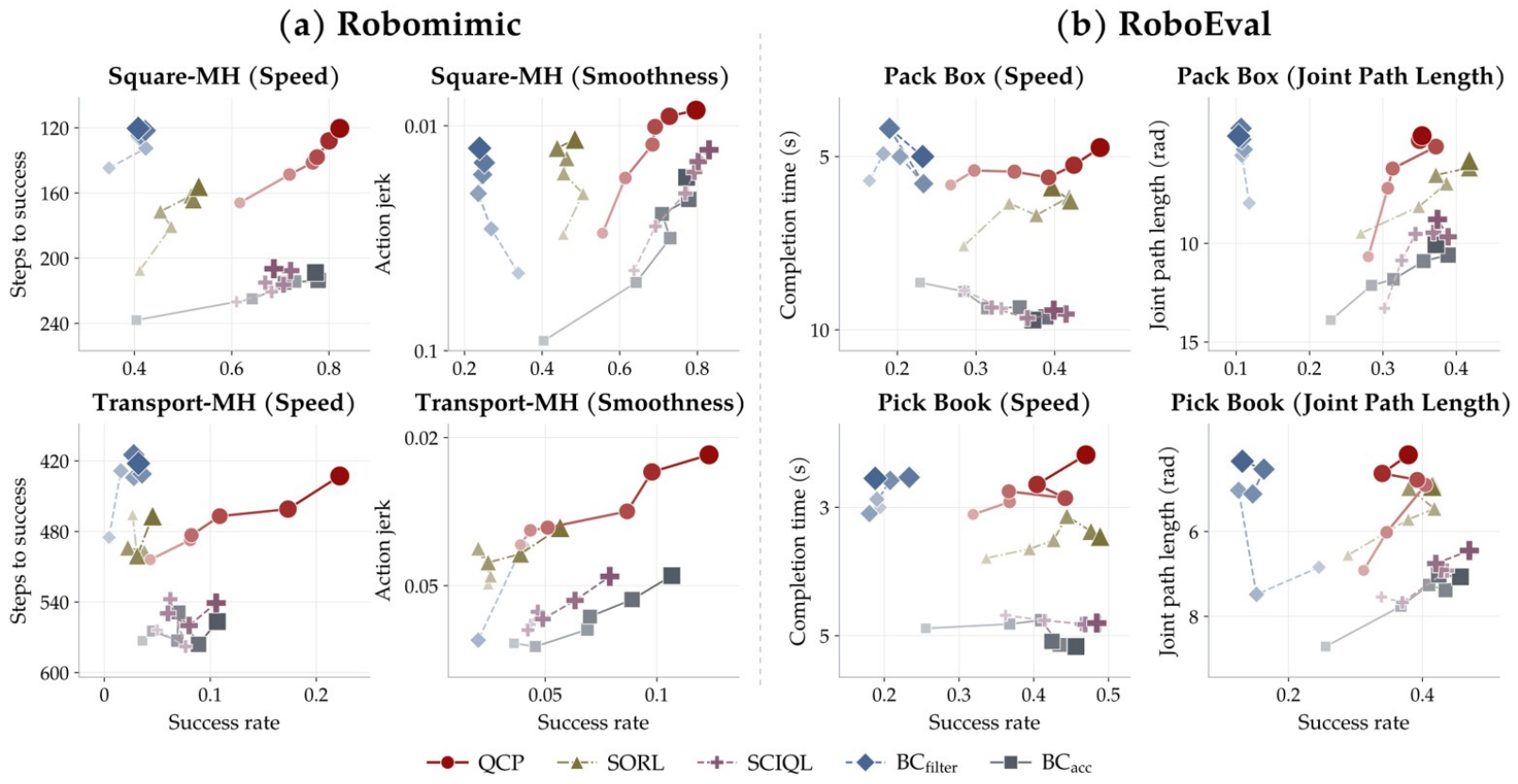
Why can rollouts exceed the best demo?

- Labels attach to each action chunk → high-quality segments from different demos can be stitched across stages
- Action sampling is stochastic; top-level conditioning → higher-quality samples are more likely

Why retrain from scratch each round?

- Relabeling shifts each label's quality range, so the old label → behavior mapping must be forgotten
- May also mitigate plasticity loss in continual training (not directly verified by the authors)

Main result: QCP alone improves both quality and success rate



Why baselines are lopsided

BC_acc

No quality signal → only success rate improves

BC_filter / SORL

Uses only label-matched data, discarding most successes → hurts success rate

SCIQL

Weights success + quality advantages → trade-off

QCP

Conditions all successes on labels → moves up and to the right

x: success rate; y: quality (up is better); darker = later iteration; 3 seeds

Vs. online/offline RL: QCP has the best quality in all 8 settings

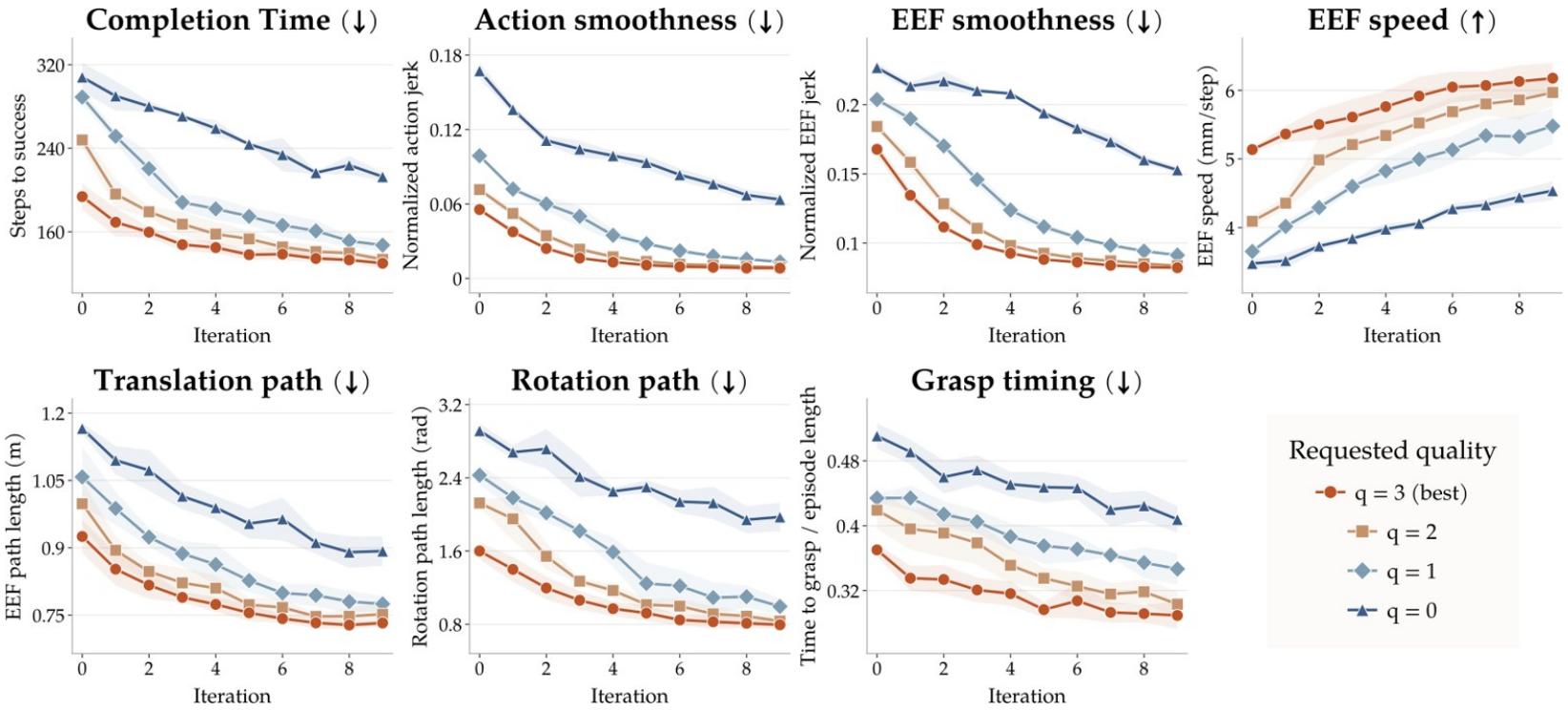
Method	Square Speed (steps)	Square jerk ×1e-3	Transport Speed (steps)	Transport jerk ×1e-3	Pack Box Speed (s)	Pack Box JPL (rad)	Pick Book Speed (s)	Pick Book JPL (rad)
QCP (ours)	**120.3** SR 82.2%	**8.5** SR 79.7%	**432.8** **SR 22.2%**	**22.2** **SR 12.3%**	**4.82** SR 45.8%	**6.42** SR 35.3%	**2.43** SR 47.0%	**4.63** SR 37.9%
SORL	156 SR 53.2%	11.5 SR 48.4%	466.8 SR 4.6%	34.9 SR 5.7%	5.64 SR 39.7%	7.11 SR 41.8%	3.37 **SR 48.9%**	5.15 SR 41.4%
SCIQL	206.5 SR 68.7%	12.8 **SR 83.0%**	540.8 SR 10.6%	47.2 SR 7.9%	9.24 SR 39.9%	9.06 SR 37.4%	4.75 SR 48.4%	6.4 **SR 47.0%**
DSRL	208.8 SR 37.0%	46.9 SR 43.0%	549.5 SR 4.0%	79.4 SR 7.0%	6.35 **SR 60.7%**	66.05 **SR 50.7%**	— SR 0.0%	— SR 0.0%
EXPO	162.5 **SR 96.0%**	157.4 SR 79.0%	567.7 SR 11.0%	176.5 SR 3.0%	8.72 SR 39.0%	24.43 SR 43.7%	5.73 SR 3.0%	29.2 SR 4.3%

Large: quality metric (lower is better; red = best); small: success rate SR (bold = column best). Iteration 5, mean of 3 seeds.

Note: EXPO on Square speed and DSRL on Pack Box reach higher SR but clearly worse quality; DSRL / EXPO largely fail on Pick Book.

Authors: RL for flow policies is tuned for success signals and struggles to exploit the added quality reward (λ=4, tuned on Square only)

Controllability holds: the 4 levels never cross over 10 iterations

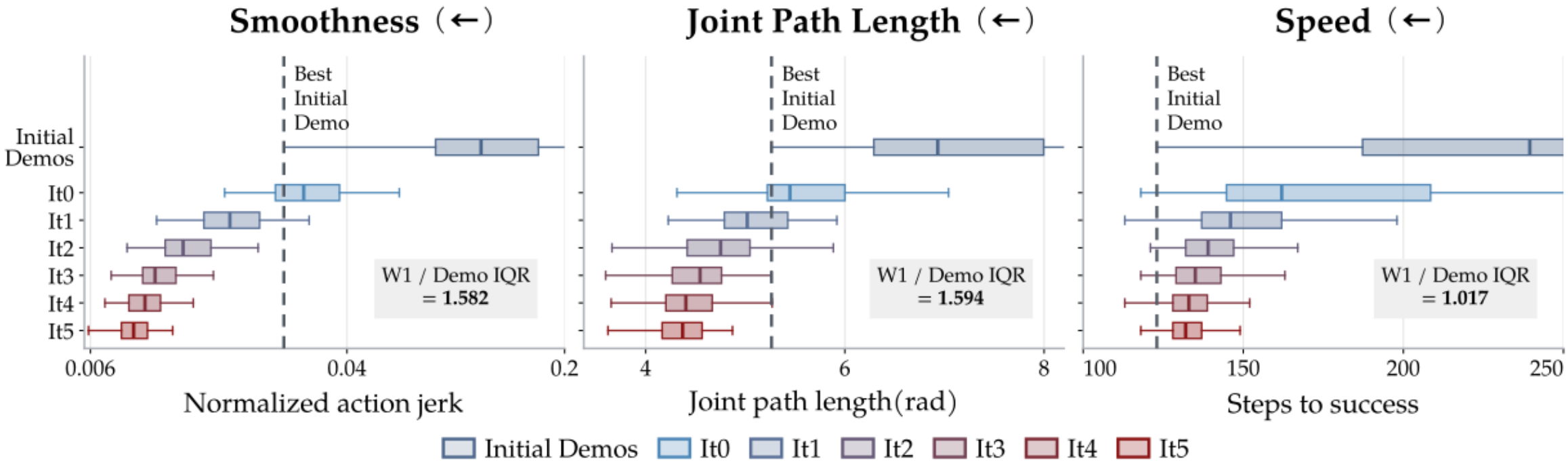


193.7 vs 308.1

Square mean steps, round 0 request q=3 vs q=0 (mid levels in between)

- New data comes only from the top level, yet the level order of all 7 metrics holds every round
- The whole controllable range shifts toward higher quality together (arrows)
- One policy: keeps improving while still offering selectable execution styles

Beyond demos: successful rollouts surpass the best initial demonstration



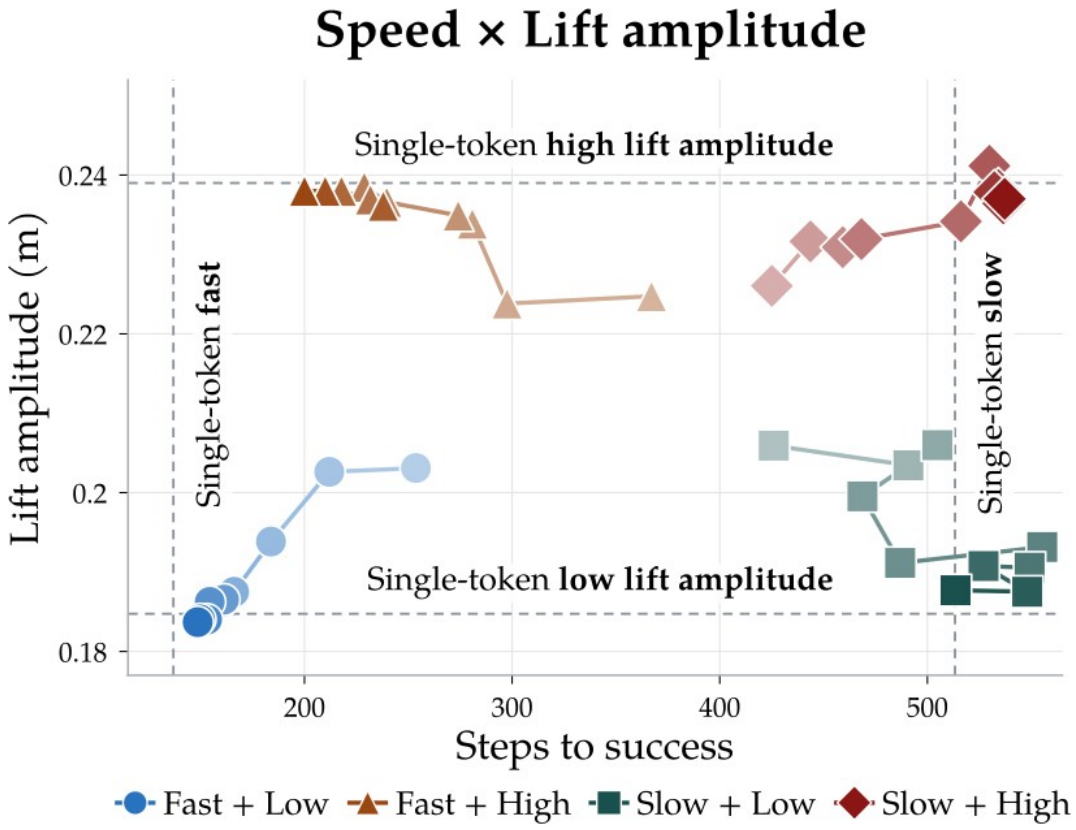
1.582
Smoothness (normalized jerk)
Square

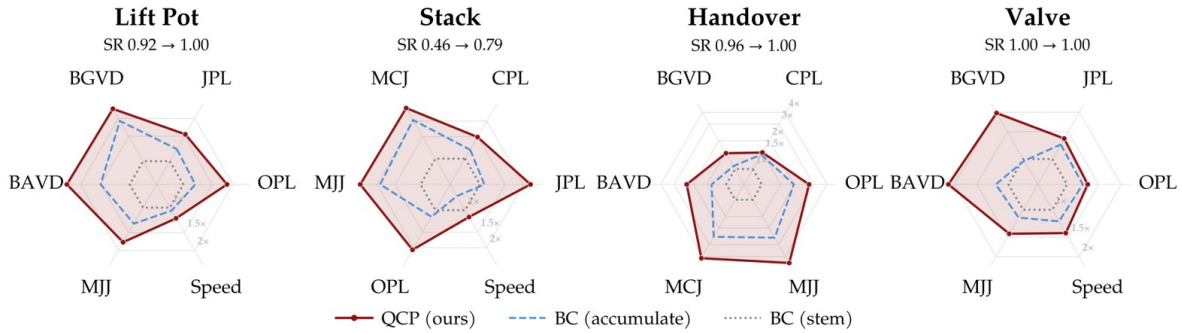
1.594
Joint path length (JPL)
RoboEval

1.017
Speed (steps to finish)
near the physical speed limit

W1 / IQR
Wasserstein-1 distance between initial demos and final successful rollouts ÷ IQR of demo scores

Multi-attribute: combined two-way control; 6 attributes improve jointly





0.46→0.79
Stack success rate

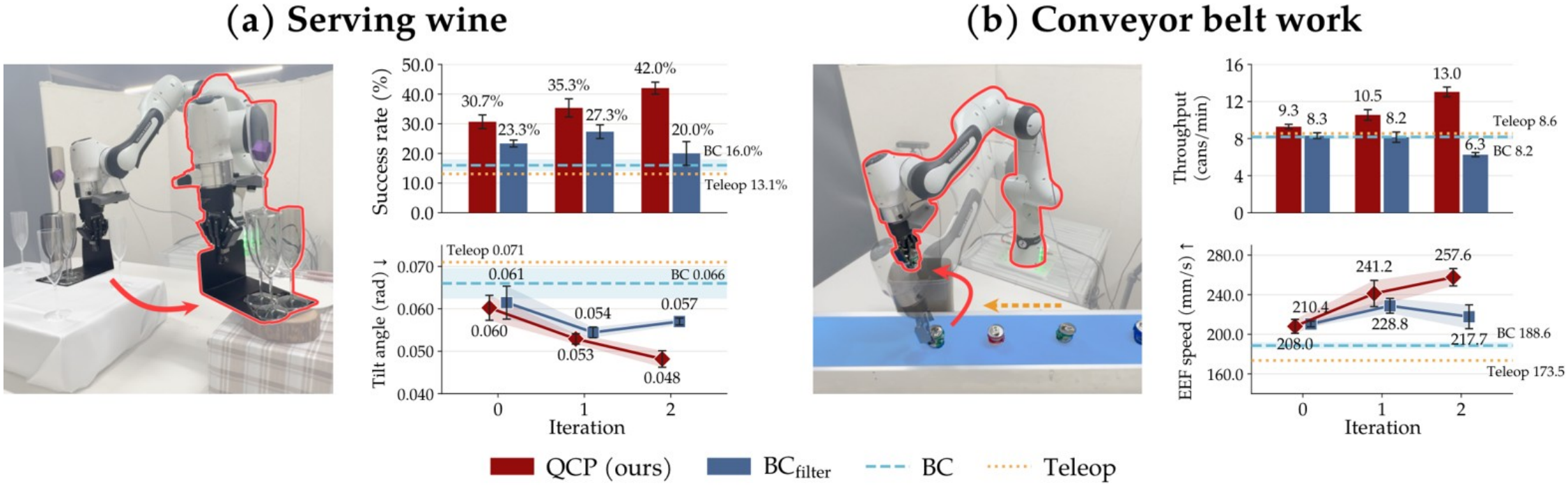
8.16→3.63
Stack joint path length

169.6→36.2
Handover joint jerk

Square, two attributes: all four fast/slow × high/low lift combos approach the single-attribute references (dashed); slow combos have lower success

RoboEval, 6 attributes: one policy conditioned on 6 quality labels; outward = better (initial BC = 1×). 2 unaffected or task-conflicting attributes excluded per task

Real robot (π0.5 + Franka): steadier tray, faster conveyor, beats teleop



42.0%
Wine-tray success rate
BC 16.0% • teleop 13.1%

−27%
Mean tray tilt
0.066 → 0.048 rad

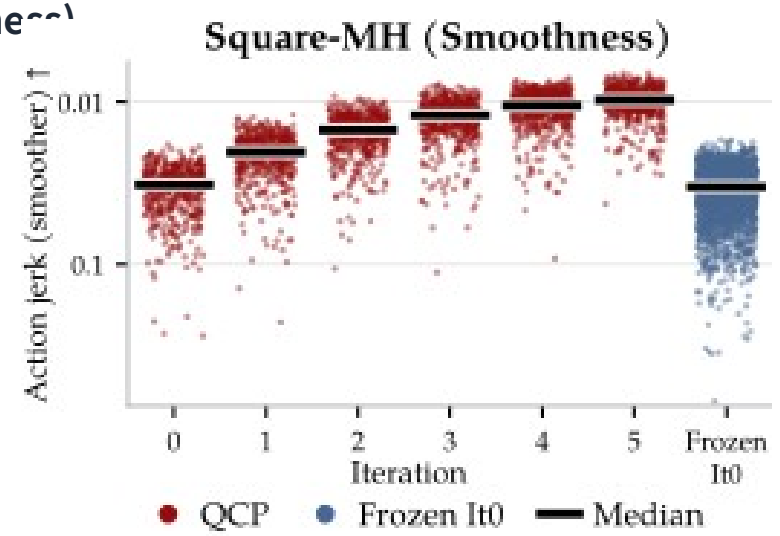
13.0
Conveyor throughput (cans/min)
BC 8.2 • +59%

257.6
EEF speed (mm/s)
BC 188.6 • teleop 173.5

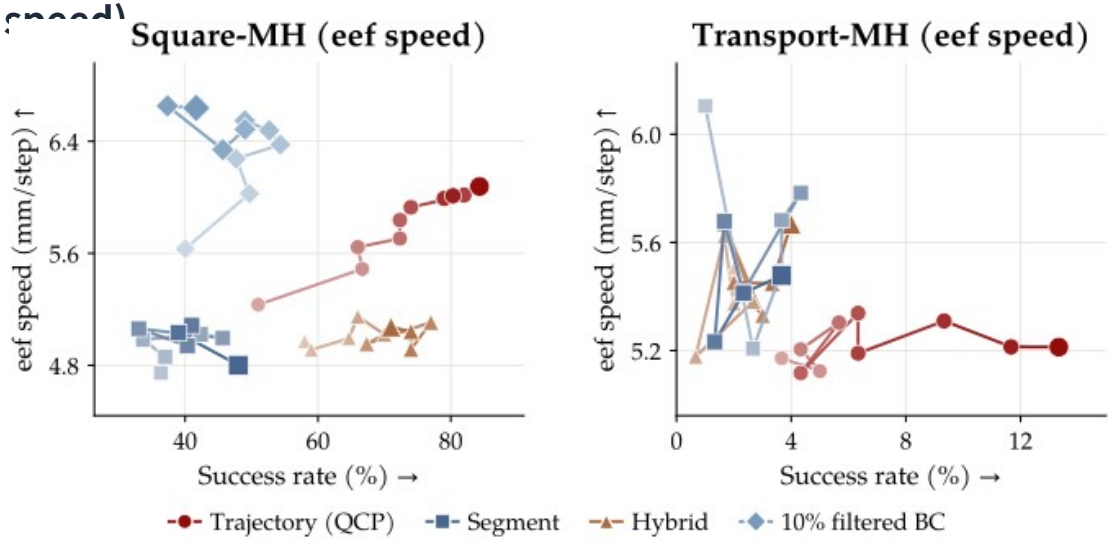
ANALYSIS

Ablation: gains come from iterating; trajectory labels beat segment labels

Same 1,800 rollouts: frozen It0 vs. iterative (Square smoothness)



Label granularity: trajectory vs. segment vs. Hybrid (EEF speed)



0.0335

Frozen It0 median jerk smoothest run 0.0171 (still worse than It5's 75th pct.)

0.0098

Iteration-5 median jerk success rate 73.9%

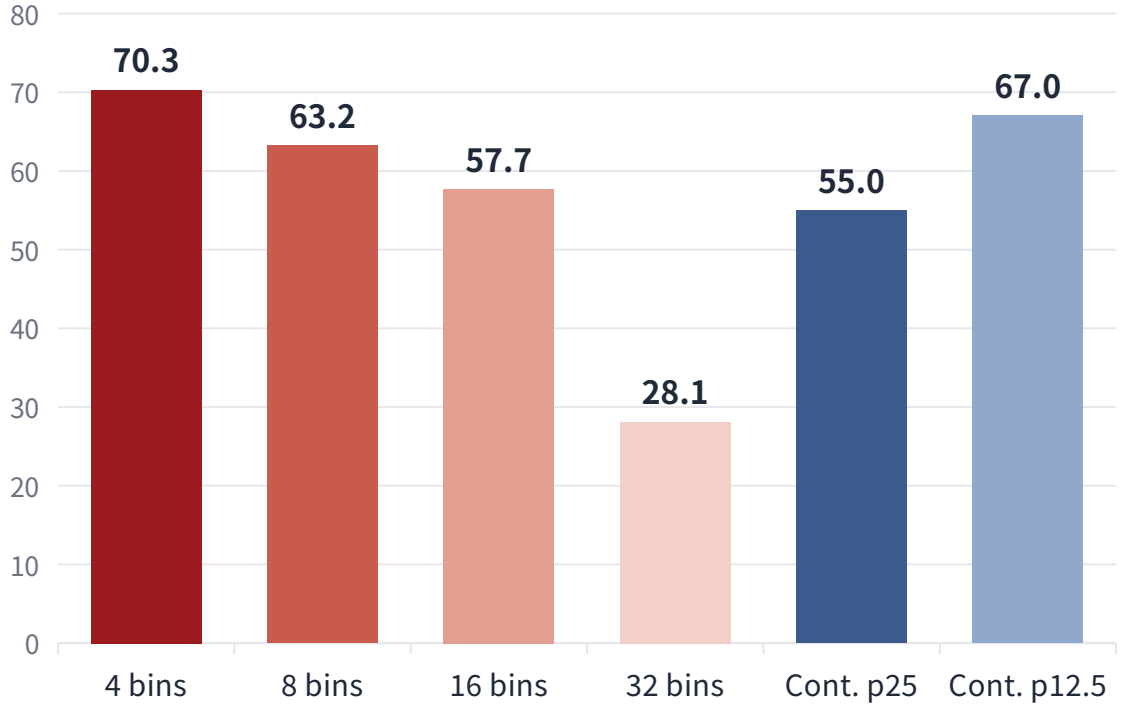
Square, round 9: trajectory **84.3% / 6.08**; segment 48.0% / 4.80; Hybrid 71.0% / 5.07 (SR / mm per step)

Transport: trajectory 13.3% vs. 3.7% / 4.0%

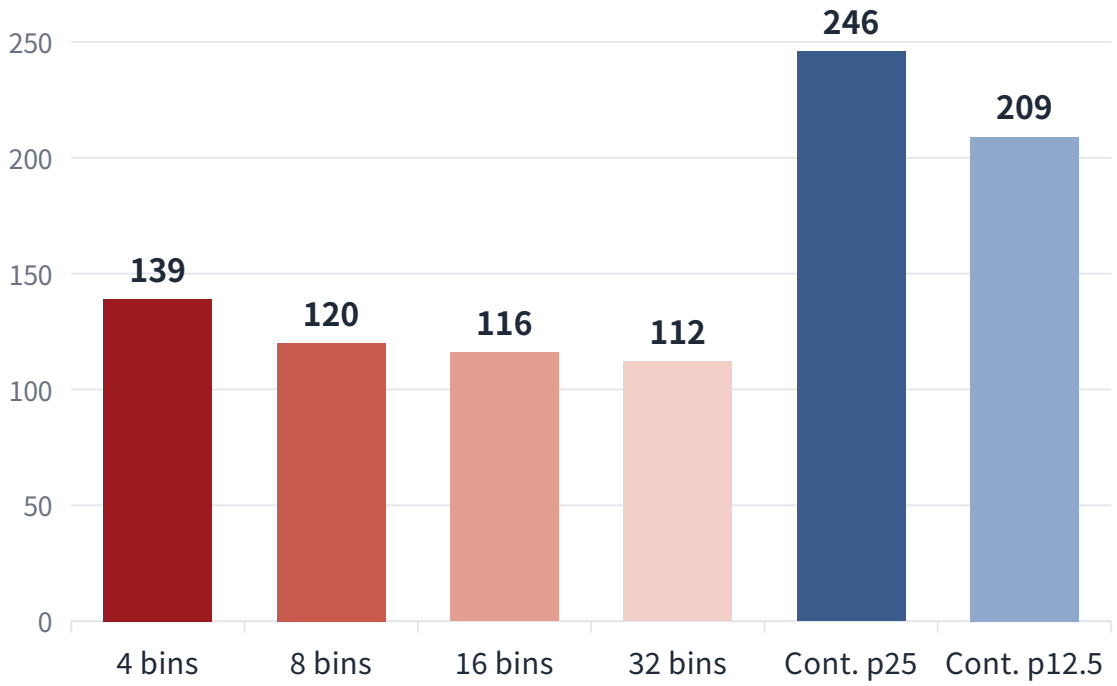
Segment scores couple with task phase; the policy inflates local scores without advancing the task (single seed)

Quartile labels are a safe default: finer bins are faster but lose success

Success rate (%, higher is better)



Steps to completion (lower is better)



Why does quantization help? With 4 bins, the top bin holds ~1/4 of training trajectories, giving ample support; a continuous target matches only a few examples. The continuous variant also picks request values differently, so the conclusion is that quartiles + per-round recomputation are a reliable default, not that continuous conditioning fails.

Source: App. F (Fig. 9 right) data, replotted here

DISCUSSION

Verdict: simple, practical; relies on success checks and hand-coded quality

Strengths

- Minimal: rule-based quality functions + success filtering suffice to bootstrap; no reward shaping or human preferences
- VLA-friendly: just add “quality level” to the prompt; no architecture changes
- “Condition, don't filter” breaks the quality–success trade-off, with strong evidence
- Thorough ablations: frozen policy, bin count, continuous conditioning, segment labels

Limitations / Questions

- Relies on success detection and predefined quality functions; subjective qualities (e.g., “elegance”) are hard to capture
- Retrains from scratch each round on ever-growing data; VLA cost grows linearly with iterations
- Only 0–2 real-robot iterations; Transport final SR only 22% (12% for smoothness)
- RL baseline coefficients tuned on Square only; multi-attribute conflicts handled by manually excluding attributes

TAKEAWAYS

Three Takeaways

01 Conditioning beats filtering

All successful data is used: low-quality trajectories teach how to succeed, labels decide how it is done → quality and success rate can rise together

02 Hindsight relabeling + quantile recomputation

With only a rule-based quality function and success detection, the policy bootstraps from its own rollouts, pushing the controllable range beyond the demos

03 Near-zero changes for VLAs

π0.5 changes only the prompt text, yet yields higher real-robot throughput (+59%) and steadier transport (tilt −27%)

When to use: existing BC/VLA policies that must switch between “fast / stable / efficient” at deployment, with automatic success detection and computable quality metrics